노디르벡 마흐투모프

NODIRBEK MAKHTUMOV

AI/ML 엔지니어 포트폴리오

컴퓨터 비전 · 엣지 AI · 모델 최적화

E-7 특정활동 · 국내 취업 가능

stark.developer2877@gmail.com · 010-5751-0750
github.com/Nodir0705 · linkedin.com/in/nodirbek-makhtumov-03084a194
nodir0705.github.io
대전광역시 · 2026년 11월

목차

재직 프로젝트

01 스마트 쓰레기 수거차량 관제 시스템 2023.04 – 현재 2

02 Edge-Aware 세그멘테이션 파이프라인 2023.09 – 현재 4

03 SAM2 기반 영상 자동 라벨링 도구 2024 6

계약 프로젝트

04 스마트 트럭 주차 관제 및 차량 ID 인식 2025.09 – 2025.11 7

05 얼굴 인식 출퇴근 키오스크 2026.07 10

06 Sales Agent (사람이 승인하는 이메일 견적) 2026.10 12

개인 프로젝트

07 이미지 기반 분류기의 강건성 연구 2026 14

08 IoT 보안 게이트웨이 (Orange Pi 5 · RK3588S) 2026 16

09 Smart Pointer (발표 음성 기반 슬라이드 하이라이트) 2026 17

기술 스택 · 학력 · 어학 · 비자 19

핵심 요약

엣지 하드웨어에서 실제로 동작하는 컴퓨터 비전 시스템을 만듭니다.

- 객체 탐지·세그멘테이션·실시간 영상 분석 기반의 프로덕션 비전 시스템을 **3년 이상** 개발·상용화했습니다.
- 모델 설계와 학습부터 **TensorRT·ONNX 양자화/프루닝**, NVIDIA Jetson·Hailo-8·RK3588 NPU 배포까지 전 과정을 직접 수행합니다.
- FastAPI 추론 서비스, PostgreSQL 메타데이터 로깅, Docker 기반 엣지 배포로 **엔드투엔드** 책임집니다.
- 정확도 수치 하나로 끝내지 않고 **클래스별 지표·적대적 강건성·하드웨어별 벤치마크**로 모델을 검증합니다.
- **사람이 승인하는 LLM 에이전트**도 개발하며, 평가 세트로 성능을 측정합니다.
- 우즈베크어(모국어)·영어·한국어·러시아어 사용, **E-7 특정활동 비자** 보유로 국내 즉시 근무 가능합니다.

30%	25+ FPS	8.6×	76% → 92%
GPU 메모리 절감	4채널 동시 처리	NPU 추론 가속	트럭 ID 인식 정확도
YOLOv8 백본 교체	실시간 세그멘테이션	Hailo-8 · 지터 95× 감소	디스큐잉 모듈 도입

01 스마트 쓰레기 수거차량 관제 시스템

YOLO 기반 폐기물 분류 및 적재량 추정 — 차량 탑재 엣지 추론

넷비젼텔레콤 · AI/ML 엔지니어 · 2023.04 – 현재

YOLOv8 | ResNet50 | TensorRT | Jetson Orin NX | DLA | Docker



> **비즈니스 임팩트**
> 메모리 확보 덕분에 차량에 탑재된 단일 Jetson 보드에서 비전 모델과 기존 관제 프로세스를 함께 상시 구동할 수 있게 되었습니다. 보드 증설 없이 기존 하드웨어 위에서 비전 기능을 추가한 것이 이 작업의 핵심 성과입니다.

문제 정의

Jetson Orin NX 환경에서 폐기물 종류 분류와 적재량 추정을 실시간으로 수행해야 했습니다. 그러나 기본 YOLOv8 모델의 GPU 메모리 사용량이 과도해, 동일 보드에서 구동되는 다른 관제 프로세스와 함께 상시 운영하기 어려웠습니다.

기술적 접근

- **백본 교체 전략** — YOLOv8 기본 백본을 ResNet50으로 교체하고, 1x1 convolution 채널 어댑터를 추가해 기존 PANet neck의 feature map 차원에 정합시켰습니다.
- **DLA 오프로딩** — ResNet 계열 레이어가 Jetson의 DLA(Deep Learning Accelerator)에서 지원되는 점을 활용해 연산을 GPU에서 DLA로 분산했습니다. 이것이 메모리 절감의 핵심이었습니다.
- **통제된 A/B 평가** — 동일 타깃 하드웨어에서 mAP·FPS·메모리 사용량을 기존 구조와 커스텀 구조 간 직접 비교했습니다.
- **최적화 및 배포** — 양자화와 TensorRT 변환으로 런타임을 추가 단축하고, 발열·전력 제약 안에서 Docker로 컨테이너화해 재현 가능한 배포를 구성했습니다.

스마트 쓰레기 수거차량 관제 시스템

아키텍처 비교

기존 구조

YOLOv8m

백본 — CSPDarknet53

Stem + Conv Layers
3 → 64ch

C2f Blocks (Stage 2–3)
64 → 192ch

C2f Blocks (Stage 4)
192 → 384ch

C2f + SPPF (Stage 5)
384 → 576ch

Neck — PANet / FPN 동일

Upsample + Concat + C2f
다중 스케일 융합

Head — Decoupled 동일

Detect (P3, P4, P5)
cls + bbox

개선 구조

YOLOv8 + ResNet50

백본 — ResNet50 (교체)

Conv7x7 + MaxPool
3 → 64ch

Layer 1–2 (Bottleneck)
64 → 512ch

Layer 3 (Bottleneck)
512 → 1024ch

Layer 4 (Bottleneck)
1024 → 2048ch

1x1 Conv 채널 어댑터 — 추가

1x1 Conv 채널 어댑터
512/1024/2048 → 192/384/576 · neck 차원 정합

DLA 오프로딩 — 핵심 최적화

DLA 오프로딩
ResNet 계열 레이어(Conv·BN·ReLU)는 DLA 지원 → GPU에서 분리

CSPDarknet의 C2f 블록은 커스텀 연산을 포함해 DLA로 완전히 지원되지 않아 GPU에서만 동작합니다.

Neck — PANet / FPN 동일

Upsample + Concat + C2f
다중 스케일 융합

Head — Decoupled 동일

Detect (P3, P4, P5)
cls + bbox

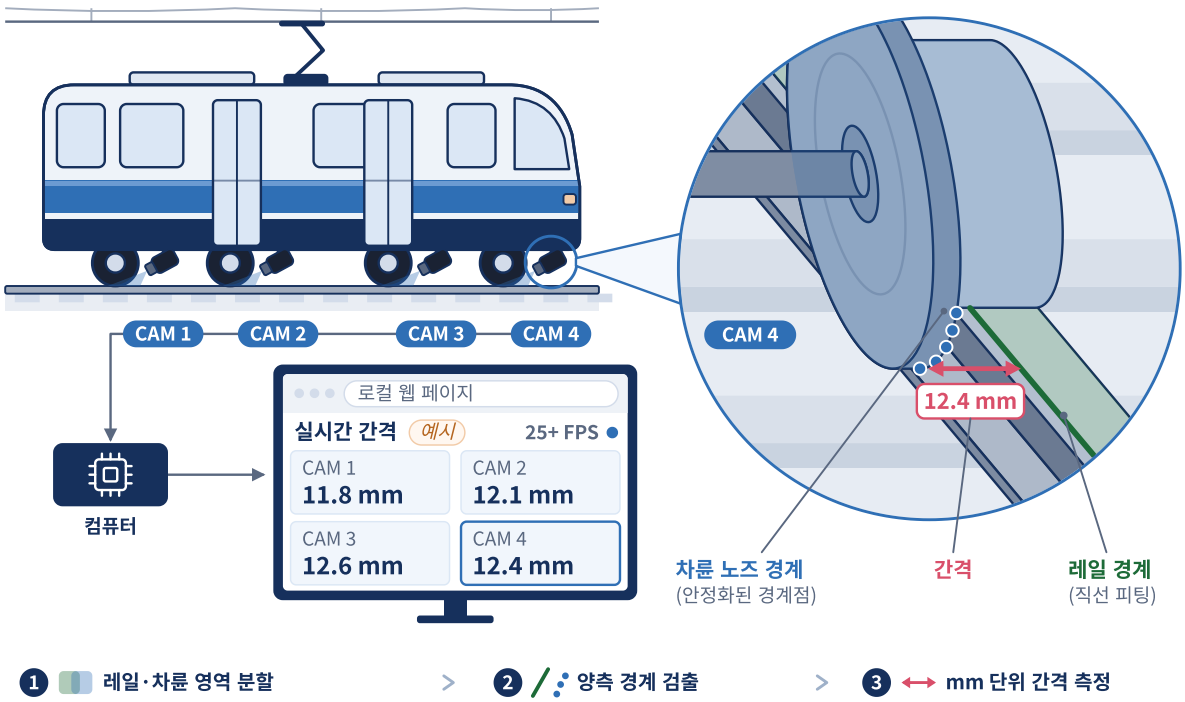
백본	핵심 변경	절감 원리	GPU 메모리
CSPDarknet → ResNet50	1x1 Conv 어댑터	ResNet 레이어 → DLA 오프로드	↓ 30%

02 Edge-Aware 세그멘테이션 파이프라인

레일-차륜 간격 실시간 측정 시스템 — 4채널 동시 처리

넷비젼텔레콤 · AI/ML 엔지니어 · 2023.09 – 현재

PyTorch | RANSAC | FastAPI | PostgreSQL | OpenCV | Multi-threading



1 레일·차륜 영역 분할 > 2 양측 경계 검출 > 3 mm 단위 간격 측정

> **비즈니스 임팩트**
> 사람이 수행하던 레일-차륜 간격 점검을 다채널 카메라 기반 자동 측정으로 대체하고, 모든 측정값을 타임스탬프와 함께 적재해 이력 추적과 사후 분석이 가능하도록 했습니다.

문제 정의

다수의 카메라 스트림에서 레일과 차륜의 경계를 실시간으로 정밀하게 검출해야 했습니다. 일반적인 세그멘테이션 모델은 특히 곡면 경계에서 노이즈가 많고 프레임마다 흔들리는 경계를 출력했고, 이 위에서 계산되는 간격 측정값을 신뢰할 수 없었습니다.

Edge-Aware 세그멘테이션 파이프라인

시스템 아키텍처

기술적 접근

- **경량 세그멘테이션 모델** — 추론 속도를 확보하면서도 경계 정밀도를 유지하도록 설계한 경량 아키텍처로 레일 영역과 차륜 영역을 동시에 분할합니다.
- **RANSAC 기반 경계 추출** — 레일 경계에 RANSAC 회귀로 기하학적 직선을 피팅해 세그멘테이션 노이즈와 이상점을 제거했습니다.
- **자체 포인트 안정화 알고리즘** — 차륜 경계점을 연속 프레임에 걸쳐 시간축으로 안정화해, 실제 위치 변화는 보존하면서 프레임 간 지터만 억제했습니다.
- **동시 스트림 처리** — 스트림별 독립 처리 큐를 갖는 멀티스레드 파이프라인으로 4개 카메라 피드를 동시에 처리합니다.
- **서빙 및 메타데이터 적재** — FastAPI 엔드포인트로 서빙하고, 추론 메타데이터와 간격 측정값을 PostgreSQL에 구조화해 기록합니다.



03

SAM2 기반 영상 자동 라벨링 도구

시간축 전파를 활용한 자동 어노테이션 파이프라인

넷비젼텔레콤 · AI/ML 엔지니어 · 2024

SAM2 | PyTorch | Video Processing | YOLO/COCO Export | OpenCV

> **비즈니스 임팩트**
> 학습 데이터셋 구축이 팀의 병목이었습니다. 수작업 어노테이션을 자동 전파로 대체해 신규 비전 과제의 데이터 준비 기간을 크게 단축했습니다.

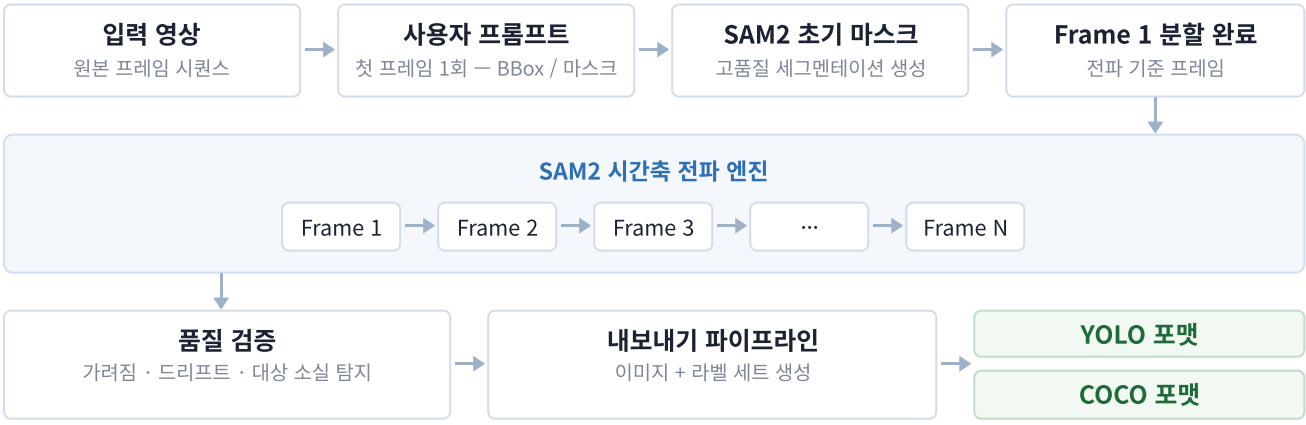
문제 정의

고품질 컴퓨터 비전 모델을 학습시키려면 일관되게 라벨링된 대규모 데이터셋이 필요합니다. 그러나 영상 어노테이션을 수작업으로 수행하는 것은 지나치게 많은 시간을 소모했고, 작업자마다 라벨 기준이 달라지는 문제도 있었습니다.

기술적 접근

- **프롬프트 기반 초기 분할** — 첫 프레임에 사용자가 바운딩 박스 또는 마스크를 한 번만 지정하면 SAM2가 고품질 초기 마스크를 생성합니다.
- **시간축 전파** — SAM2의 temporal propagation으로 이후 모든 프레임에서 대상을 추적·분할해 영상 전체에 일관된 라벨을 자동 부여합니다.
- **라벨 일관성 검증** — 가려짐(occlusion)이나 대상 소실로 인한 전파 실패를 후처리 단계에서 탐지해 걸러냅니다.
- **학습 포맷 내보내기** — YOLO와 COCO 포맷으로 자동 변환해 기존 학습 프레임워크에 바로 투입할 수 있도록 했습니다.

파이프라인 구조



성과

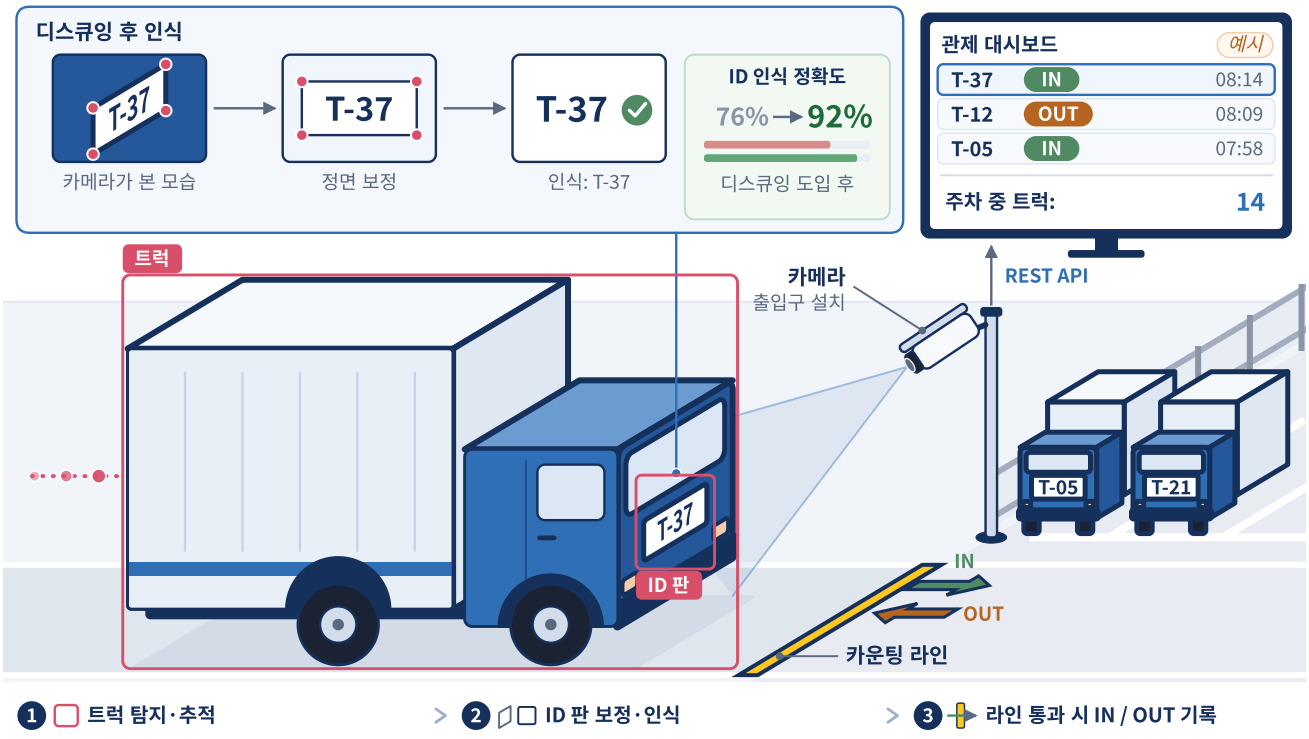
1 프레임	영상 전체	10배+	YOLO · COCO
수작업 입력	자동 라벨링	작업 속도 향상	내보내기 지원

04 스마트 트럭 주차 관제 및 차량 ID 인식

단독 구축한 엔드투엔드 실시간 비전 시스템

프리랜서 계약 프로젝트 · AI/ML 엔지니어 · 2025.09 – 2025.11 ·
github.com/Nodir0705/smart-truck-parking

YOLO | ByteTrack | EasyOCR | OpenCV | Perspective Transform | REST API



> **비즈니스 임팩트**
> 주차장 출입 확인을 수기 대장에서 자동 인식으로 전환하고, 실시간 점유 현황을 관제 화면에 제공해 운영자가 현장을 돌지 않고도 상태를 파악할 수 있게 했습니다.

문제 정의

트럭 주차장에서 차량 탐지, 고유 ID 인식, 실시간 점유 현황 스트리밍을 자동화해야 했습니다. 실제 설치 환경에서는 ID 판이 비스듬한 각도로 촬영되고 조명 조건도 좋지 않아, 초기 OCR 인식률이 76%에 그쳤습니다.

스마트 트럭 주차 관제 및 차량 ID 인식

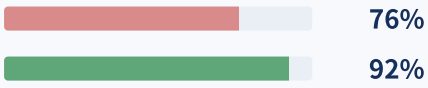
기술적 접근

- **다단계 탐지 구조** — YOLO 기반 탐지로 트럭 위치와 ID 판 ROI를 각각 찾아내고, ID 영역만 잘라내어 인식 단계로 전달합니다.
- **추적과 ID 확정** — ByteTrack으로 트럭마다 트랙을 유지하고, 라인 통과 시점에 IN / OUT 이벤트를 생성합니다. 트럭 ID는 최근 5회 인식 결과를 투표해 확정합니다.
- **자체 디스큐잉 파이프라인** — contour 검출과 adaptive threshold로 ID 판의 네 꼭짓점을 찾고 perspective transform과 회전 보정으로 기하 왜곡을 정면으로 보정하는 모듈을 직접 구현했습니다.
- **EasyOCR 연동** — 해당 트럭 ID 포맷에 맞춰 조정하고, 대비 향상과 노이즈 제거 전처리를 앞단에 배치했습니다.
- **실시간 스트리밍** — 입출차 이벤트와 인식된 ID를 REST API로 백엔드에 실시간 전송해 관제 대시보드에 반영합니다.
- **체계적 오류 분석** — 실패 사례를 유형별로 분류한 결과 원근 왜곡이 주요 실패 원인임을 특정했고, 이것이 디스큐잉 모듈 도입의 근거가 되었습니다.
- **오픈소스 버전** — 디스큐잉 모듈을 새로 작성한 오픈소스 버전을 공개했습니다. 합성으로 만든 기울어진 ID 판 200장에서 EasyOCR 정확도가 80.5% → 97.5%로 올랐습니다. 고객사 현장 수치(76% → 92%)와는 별개의 측정입니다.

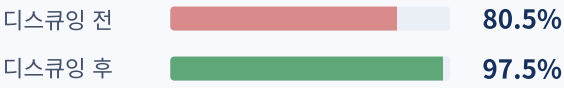
성과

76% → 92%	실시간	엔드투엔드	자체 개발
ID 인식 정확도 (현장)	백엔드 스트리밍	단독 구축	디스큐잉 알고리즘

고객사 현장 (납품 시스템)
현장 카메라 기준 ID 인식 정확도



오픈소스 버전 (별도 측정)
합성 기울어진 ID 판 200장 · EasyOCR 정확도



체계적 오류 분석으로 원근 왜곡이 주요 실패 원인임을 특정한 뒤 디스큐잉 모듈을 도입했습니다. 두 결과는 데이터와 구현이 달라 서로 직접 비교하지 않습니다.

스마트 트럭 주차 관제 및 차량 ID 인식

파이프라인 구조



05 얼굴 인식 출퇴근 키오스크

파이프라인 3종 실측 비교를 거쳐 Samsung Galaxy Tab 온디바이스 추론으로 납품

프리랜서 계약 프로젝트 · AI/ML 엔지니어 · 2026.07 · **github.com/Nodir0705/face-recognition**

SCRFD | ArcFace | InsightFace | Hailo-8 NPU | Raspberry Pi 5 | C++ | Galaxy Tab | TFLite | SQLite

> **비즈니스 임팩트**
> 하드웨어를 추측으로 정하지 않고, 같은 파이프라인을 세 가지로 구현해 실측한 결과로 선정했습니다. 최종적으로 Samsung Galaxy Tab에서 온디바이스로 동작하는 출퇴근 키오스크를 납품해, 별도 추론 장비 없이 태블릿 한 대로 운영됩니다.

문제 정의

실시간 얼굴 인식 출퇴근 키오스크를 구축하되, 어떤 하드웨어에 올릴지가 정해지지 않은 상태였습니다. CPU 추론은 응답 속도 자체도 느렸지만 더 큰 문제는 지연 시간의 편차였습니다. 사용자가 카메라 앞에서 얼마나 기다려야 하는지 예측할 수 없으면 키오스크는 느리게 느껴집니다.

기술적 접근

- **하드웨어 선정을 위한 3종 실측** — 같은 인식 파이프라인을 Python/CPU, C++ 포팅, Hailo-8 NPU 세 가지로 각각 구현해 직접 측정했습니다. Python/CPU와 Hailo-8은 같은 Raspberry Pi 5에서 동일 이미지로 비교했고, C++ 포팅은 Raspberry Pi 4에서 Python 버전과 비교했습니다.
- **모델 구성** — SCRFD로 얼굴을 검출하고 ArcFace 512차원 임베딩(InsightFace `buffalo_sc`)을 코사인 유사도로 대조합니다. 3프레임 연속 일치와 눈 깜빡임·움직임 라이브니스 검사를 통과해야 출퇴근이 기록됩니다.
- **가이드 등록 마법사** — iPhone Face ID 방식을 참고해 정면·좌·우·상·하 5개 자세를 안내하고, 각 자세를 안정적으로 유지하면 자동 촬영합니다.
- **오프라인 우선 설계** — SQLite(WAL 모드)에 먼저 기록하고 백그라운드에서 Google Sheets로 동기화해, 네트워크가 끊겨도 출퇴근 기록이 유실되지 않습니다.
- **Galaxy Tab 온디바이스 납품** — 실측 결과를 근거로 최종 배포 대상을 Samsung Galaxy Tab으로 결정하고, Kotlin·CameraX·TFLite로 태블릿 자체에서 추론이 동작하도록 이식했습니다. 현장에는 태블릿 한 대만 설치됩니다.
- **관리 도구** — Flask 웹 앱과 관리자 대시보드를 함께 제공하고, GitHub Actions로 테스트를 자동화했습니다.

얼굴 인식 출퇴근 키오스크

인식 파이프라인 · 벤치마크 실측 결과



Python vs Hailo-8 NPU

Raspberry Pi 5 · 동일 이미지 · 얼굴 1건당 검출+임베딩 기준 (마지막 열은 얼굴 5개가 있는 한 프레임)

파이프라인	중앙값 지연	지터	처리량	얼굴 5개 프레임
Python + CPU (4 threads)	48.08 ms	±27 ms	~21 fps	124.21 ms
Hailo-8 NPU	**5.59 ms**	**±0.29 ms**	**~180 fps**	**20.37 ms**

> 지연 시간 8.6배 단축보다 **지터가 95배 줄어든 것**이 실제 체감에서 결정적이었습니다. 평균이 빨라도 편차가 크면 키오스크는 느리게 느껴집니다.

Python vs C++ 포팅

Raspberry Pi 4 · 검출+인식 1회 기준 (위 Pi 5 결과와 별도 측정)

구현	검출+인식 1회	루프 오버헤드
Python	~430 ms	~10 ms
C++ 포팅	~410 ms	~1 ms

> C++ 포팅은 루프 오버헤드만 줄였을 뿐, 전체 시간은 모델 연산이 좌우했습니다. 언어를 바꾸는 것만으로는 부족했고, 결과를 바꾼 것은 NPU였습니다.

성과

8.6×	95×	3종	Galaxy Tab
NPU 추론 가속	지터 감소	파이프라인 실측 비교	온디바이스 납품

06 Sales Agent

이메일 견적 에이전트 — 가격은 Python, 회신은 Claude, 승인은 대표

프리랜서 계약 프로젝트 · AI/ML 엔지니어 · 2026.10 · **github.com/Nodir0705/sales-quote-agent**

Python | Claude API | Structured Outputs | Prompt Caching | Batch API | Telegram Bot API | SQLite



비즈니스 임팩트

가격 문의에서 반복되는 작업은 에이전트가 맡습니다. 메일을 읽고, 회사 단가표로 견적을 계산하고, 회신을 작성합니다. 대표는 Telegram에서 카드를 확인하고 승인 또는 거절을 누르며, 승인 전에는 어떤 회신도 고객에게 나가지 않습니다. 구축을 마치고 실제 메일함과 Telegram에서 테스트 모드로 구동했습니다.

문제 정의

고객사는 단열 샌드위치 패널 수출 업체입니다. 영업 메일함에 들어오는 메일은 대부분 같은 유형이었습니다. 어떤 패널을 어디까지 보내면 가격이 얼마인지 묻는 문의입니다. 메일마다 Excel 단가표를 열고, 할인 구간을 적용하고, 주문량이 컨테이너 몇 대 분량인지 계산하고, 운임을 더한 뒤 영어나 러시아어로 정중한 회신을 써야 했습니다. 저는 두 가지 원칙을 중심으로 에이전트를 만들었습니다. 가격은 모델이 아니라 단가표에서 나오고, 모든 회신은 발송 전에 대표가 확인한다는 것입니다.

Sales Agent

기술적 접근

- **AI 호출 전 필터링** — 반송 메일, 자동 응답, 뉴스레터, 자기 메일함에서 보낸 메일, 무시 목록에 있는 발신자, 발신자별 한도를 넘긴 발신자는 유료 호출 전에 걸러냅니다.
- **AI가 메일을 고정 스키마로 추출** — 첫 번째 Claude 호출이 제품, 수량, 목적지, 고객의 질문, 언어를 구조화 출력으로 반환합니다. 메일 본문은 신뢰할 수 없는 데이터로 표시해 전달합니다.
- **가격은 Python이 계산** — 정확한 십진 연산으로 각 제품과 목적지를 Excel 단가표에서 찾아 할인 구간을 적용하고, 컨테이너 수를 세고, 운임을 더합니다. 제품이 모호하거나 목적지를 알 수 없으면 가격을 추측하지 않고 확인을 요청합니다.
- **모델은 가격을 직접 쓰지 않음** — 두 번째 호출은 플레이스홀더를 둔 채 회신을 작성하고, 그 자리에 Python이 직접 만든 가격 블록을 넣습니다. 이어서 Python 가드가 초안을 검사하며, Python이 제공하지 않은 금액이나 기간이 들어 있으면 발송을 차단합니다.
- **승인 없이는 발송 없음** — 대표는 계산 내역, 검사 결과, 발송될 회신 원문이 담긴 Telegram 카드를 받습니다. 승인은 데이터베이스에서 한 번만 처리되므로 두 번 누르거나 재시작해도 중복 발송되지 않으며, 수신자는 초안을 만들 때 고정됩니다.
- **호출별 비용 기록, 프레임워크 없음** — 모든 AI 호출의 비용을 계산해 저장하고, 일일 비용 상한과 발신자별 한도를 둡니다. 프롬프트는 캐싱하고, 밤사이 들어온 문의는 Batch API로 처리합니다. 에이전트 프레임워크 없이 Python 루프 하나와 SQLite 파일 하나로 구성해, 다음 개발자도 쉽게 읽을 수 있습니다.

성과

26/26	26/26	1,513개	승인
추출 평가	회신 작성 테스트 검사	자동 검사	모든 회신에 필수
모든 항목 정확	유료 API 실행	모두 통과	대표의 Telegram 카드

고객사 메일이 아닌 직접 작성한 테스트 메일 기준 · 현황: 실제 메일함과 Telegram에서 테스트 모드로 구동

테스트	검증 내용	결과
추출 평가	영어·러시아어로 작성한 문의, 후속 메일, 질문, 무관한 메일. 모든 항목이 정확해야 통과	26/26
유료 회신 작성 테스트	실제 Claude API 호출로 영어·러시아어 테스트 메일에 회신 작성, 프롬프트 인젝션 메일 포함	26/26 검사
자동 검사	가격 계산, 데이터베이스, 가드, Telegram. 외부 서비스는 모두 가짜(fake)로 대체하고 GitHub Actions에서 실행	1,513개 통과

한계: 가드는 패턴 검사일 뿐 증명이 아닙니다("몇 주"처럼 숫자가 없는 기간은 잡아내지 못함). 그래서 모든 회신을 사람이 승인합니다. 회신은 영어와 러시아어만 지원합니다. 추출 평가는 실제 고객사 메일이 아닌 직접 작성한 테스트 메일을 사용했습니다.

07 이미지 기반 분류기의 강건성 연구

분류기를 만든 뒤, 그 정확도를 믿을 수 있는지 검증하는 데 더 많은 시간을 쓴 기록

개인 프로젝트 · MalImg 데이터셋 (25개 계열 · 약 9,000 샘플) · 2026 ·
github.com/Nodir0705/malware_classifier

ResNet50 | PyTorch | PGD | Adversarial Robustness | Grad-CAM | MalConv | FastAPI | Docker

> **비즈니스 임팩트**
> 정확도 지표 하나만으로 모델을 판단하면 안 된다는 것을 정량적으로 보여준 프로젝트입니다. 실서비스 모델을 검증할 때 어떤 항목을 확인해야 하는지에 대한 제 기준이 여기서 만들어졌습니다.

문제 정의

악성코드 바이너리를 바이트 단위로 이미지화해 계열을 분류하는 모델을 만들었습니다. 테스트셋 정확도는 98.95%였습니다. 그 다음 질문은 이것이었습니다 — 이 수치를 실제로 믿을 수 있는가?

기술적 접근

- **기본 구성** — 바이너리의 각 바이트를 회색조 픽셀 하나로 변환하는 모듈을 직접 구현하고(공개 데이터셋은 이미지만 제공), ResNet50에 25-way 분류 헤드를 얹었습니다.
- **클래스 가중 손실** — 계열별 샘플 수가 약 3,000건에서 80건까지 불균형했습니다. 가중치를 적용하자 `Autorun.K` 계열의 **F1이 0 → 1.0**으로 바뀌었고, 전체 정확도는 거의 변하지 않았습니다. 전체 정확도만 봤다면 놓쳤을 문제입니다.
- **전이학습 분석** — fine-tuning 동안 백본 가중치의 변화량은 **1% 미만**(상대 L2)이었습니다. 성능 향상은 사실상 새 분류 헤드에서 나왔고, ImageNet 특징이 바이트플롯에 거의 그대로 전이된다는 뜻입니다.
- **강건성 검증** — 별도 데이터 분할과 별도 기준 모델로 재현 가능한 PGD 공격·적대적 재학습 실험을 구성하고, 덮어쓰기 공격으로 아키텍처 3종을 비교했습니다. 결과는 다음 페이지에 정리했습니다.
- **해석 가능성과 배포** — Grad-CAM으로 모델이 파일의 어느 구간을 보는지 확인하고, FastAPI 웹 앱·CLI·Docker로 분류기를 패키징했습니다.

성과

98.95%	22% → 94.15%	3종	Grad-CAM
분류 정확도	PGD 공격 시 정확도	아키텍처 비교	해석 가능성 검증
배포 분류기 · 테스트셋	강건성 실험 · 별도 기준 모델	정확도 vs 강건성	모델이 보는 구간 확인

이미지 기반 분류기의 강건성 연구

강건성 검증 결과

적대적 공격에 대한 검증 — 재현 가능한 실험

배포 분류기(98.95%)와는 별도로, 저장소에 재현 가능한 강건성 실험을 구성했습니다. 데이터 분할이 다르고(테스트셋 957개) 기준 모델도 따로 학습한 ResNet50입니다(정상 정확도 99.48%). 각 바이트 값을 최대 2만큼만 바꾸는 PGD-20 공격(ε=2/255)에 기준 모델의 정확도는 22.15%로, 최대 4(ε=4/255)에서는 6.58%로 떨어졌습니다.

모델	정상 입력	PGD-20 ε=2/255	PGD-20 ε=4/255
기준 모델 (fine-tuned ResNet50)	99.48%	22.15%	6.58%
적대적 재학습 (전체 네트워크, PGD ε=2/255)	**97.18%**	**94.15%**	**65.20%**

> 전체 네트워크를 PGD(ε=2/255) 샘플로 재학습하자 공격 상황 정확도가 **94.15%**로 회복됐고, 대가는 정상 정확도 **2.3포인트**였습니다. 다만 학습 때의 두 배 강도(ε=4/255)에서는 65.20%에 그쳐, 강건성이 더 강한 공격까지 온전히 이어지지는 않았습니다.

아키텍처별 정확도-강건성 트레이드오프

모델	정확도	파일 50% 덮어쓰기 공격 시 잔존율
ResNet50 (이미지 기반)	98.95%	43.50%
MalConv (바이트 1D CNN)	94.03%	64.09%
Byte-level Transformer	96.06%	18.27%

> 가장 정확한 모델이 가장 강건한 모델은 아니었습니다. MalConv는 정확도가 가장 낮지만 덮어쓰기 공격에는 가장 잘 버텼습니다 — 파일 전체가 아니라 핵심 바이트 패턴 하나에 집중하는 구조이기 때문입니다.

기대와 다른 결과 — 소수 샘플 신규 계열 탐지

새로운 악성코드 계열이 소수 샘플만 있을 때 이를 인식할 수 있는지 확인하려고 metric learning을 적용했습니다. 결과는 **오히려 악화**였습니다.

	학습 없는 ImageNet 특징	직접 학습시킨 버전
신규 계열 정확도 (5-way 5-shot)	98.22%	89.40%

> 원인은 학습이 관련 계열들을 특징 공간에서 서로 가깝게 압축했기 때문입니다. 기존 계열의 변종이 새로 들어오면 친척 계열들과 뭉쳐 구분이 불가능해졌습니다. **결론: 신규 변종 탐지에는 학습시키지 않은 ImageNet 특징이 더 낫습니다.** 의도한 결과는 아니었지만, 이 실험이 없었다면 잘못된 구조를 채택했을 것입니다.

한계: 위 수치는 MalImg 데이터셋 기준이며 비교적 오래된 Windows 악성코드로 구성되어 있습니다. 최신 악성코드에 대한 일반화는 검증하지 않았습니다.

08 IoT 보안 게이트웨이

클라우드 없이 보드 한 대에서 4계층 탐지부터 자동 격리까지

개인 프로젝트 · Orange Pi 5 (RK3588S) · 2026 · **github.com/Nodir0705/iot-gateway-security**

XDP/eBPF | Suricata | RK3588 NPU | RKNN | ONNX Runtime | Scikit-Learn | nftables | MQTT

> **비즈니스 임팩트**
> 클라우드 없이 보드 한 대에서 탐지부터 격리까지 동작함을 47분 스트레스 테스트로 확인했습니다.

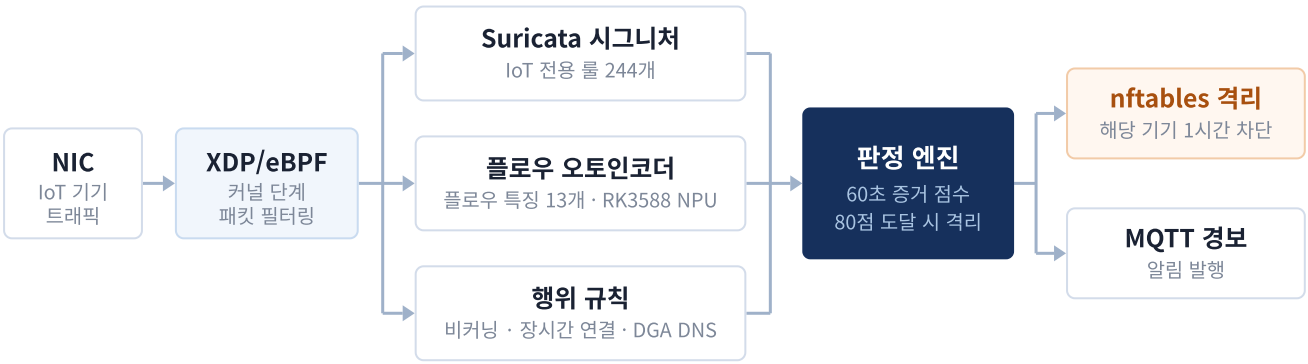
문제 정의

IoT 기기 트래픽을 클라우드 없이 게이트웨이 한 대에서 감시하고, 공격으로 판단되면 해당 기기를 자동 격리해야 했습니다. 알려진 공격과 새로운 이상 행위를 함께 다루려면 성격이 다른 탐지 계층을 하나의 판정으로 묶어야 했습니다.

기술적 접근

- **커널 단계 필터링** — XDP/eBPF로 NIC에 들어온 패킷을 커널 단계에서 먼저 필터링합니다.
- **IoT 전용 Suricata 룰** — IoT 공격을 겨냥한 Suricata 룰 244개를 직접 작성했습니다.
- **NPU 플로우 오토인코더** — 13개 플로우 특징을 입력으로 하는 오토인코더를 RK3588 NPU에서 실행합니다. 추론은 RKNN(NPU) → ONNX(CPU) → scikit-learn 랜덤 포레스트 순으로 폴백합니다.
- **행위 규칙** — 비커닝, 장시간 연결, DGA 도메인 DNS 질의를 규칙으로 탐지합니다.
- **증거 점수 기반 격리** — 계층별 탐지 결과를 점수로 누적해 60초 안에 80점에 도달하면 nftables로 해당 기기를 1시간 자동 격리하고 MQTT로 경보를 보냅니다.

파이프라인 구조



성과 · 47분 스트레스 테스트, 공격 9단계

9/9	87/88 (98.9%)	244개	1,551건
공격 단계 탐지	기대 Suricata 룰 발동	IoT 전용 Suricata 룰	경보 기록

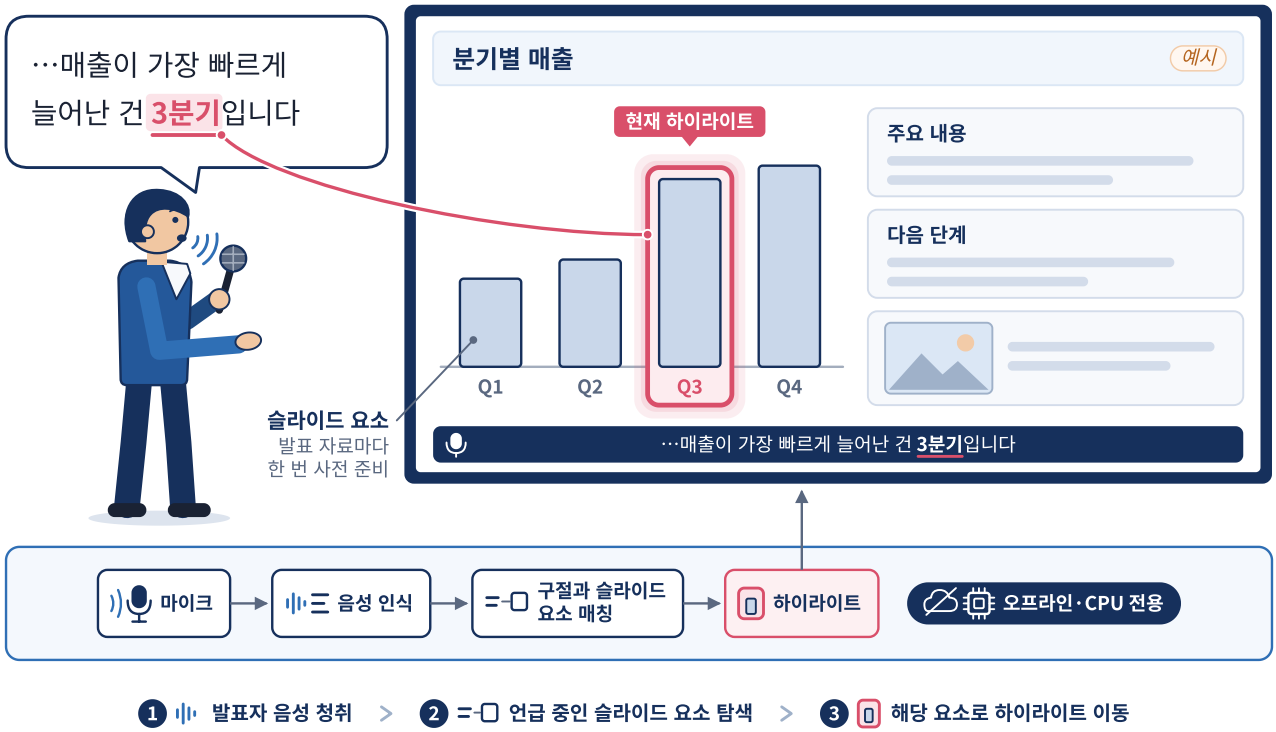
한계: 실험실 환경 전용이며, 탐지 모델은 NSL-KDD와 합성 플로우로 학습했습니다. 실제 운영 네트워크에서는 검증하지 않았습니다.

09 Smart Pointer

발표자가 말하는 내용을 따라가는 슬라이드 하이라이트

개인 프로젝트 · 2026 · **github.com/Nodir0705/smart-pointer**

Python | sherpa-onnx | Canary 180M Flash | TinyBERT Cross-Encoder | python-pptx | Claude API



비즈니스 임팩트

발표자가 말하는 동안 지금 이야기하고 있는 슬라이드 영역이 하이라이트되어, 청중은 각 문장이 무엇을 가리키는지 바로 알 수 있습니다. 라이브 루프는 CPU에서 오프라인으로 동작하며, 네트워크는 발표 자료를 준비할 때 한 번만 필요합니다.

문제 정의

발표자는 한 번에 슬라이드의 한 부분씩 설명합니다. 발표가 진행되는 동안 문장을 하나씩 듣고, 그 문장이 어떤 상자·카드·차트 막대·표 셀에 관한 것인지 판단해 그곳으로 하이라이트를 옮기는 도구를 만들고 싶었습니다. 또한 발표자가 슬라이드와 관계없는 이야기를 할 때는 아무것도 하이라이트하지 않아야 합니다.

Smart Pointer

기술적 접근

- **발표 자료별 1회 준비** — .pptx를 python-pptx와 원본 슬라이드 XML로 파싱하고 렌더링합니다. 텍스트 상자는 PDF 레이어에서 정확히 가져오고, 그림 속 텍스트는 OCR로 읽습니다. 이어서 모델이 슬라이드 맵을 작성합니다. 발표자가 가리킬 만한 요소와, 각 요소에 대해 발표자가 할 법한 짧은 표현을 정리한 것입니다. 슬라이드 맵은 Claude API로 작성하므로(로컬 Qwen 모델로 폴백) 네트워크는 준비 단계에서 한 번만 필요합니다.
- **CPU 라이브 루프** — 마이크 → 음성 인식(Canary 180M Flash) → 구절 분할 → 2계층 TinyBERT cross-encoder가 각 구절을 슬라이드 요소마다 채점 → 포인팅과 유지 → 브라우저 하이라이트 순으로 처리합니다.
- **포인팅과 유지** — 포인터는 단어마다가 아니라 구절마다 한 번 판단합니다. 고정된 "없음" 기준선이 있어 발표자가 슬라이드와 관계없는 이야기를 할 때는 아무것도 켜지 않고, 새 구절이 다른 곳을 분명히 가리킬 때까지 기존 선택을 유지합니다.
- **결정론적 동작** — CPU 스레드 하나와 고정된 모델 버전을 사용하고, 라이브 루프에서는 네트워크를 쓰지 않습니다. 그래서 같은 오디오에서는 항상 같은 하이라이트가 나오고, 녹음된 발표로 모든 변경 사항을 정확히 비교할 수 있습니다.
- **평가 벤치** — 발표 스크립트는 슬라이드를 순서대로 읽거나, 이리저리 건너뛰거나, 의미로 대상을 가리키며, 모든 문장에 해당 요소를 라벨로 달았습니다. 합성 음성(TTS)이 스크립트를 읽고, 그 오디오를 실시간보다 빠르게 음성 인식과 포인터에 통과시킵니다.
- **홀드아웃 발표 자료** — 공개 발표 자료 7종을 내려받아 테스트 전에 고정했습니다. 개발 중에는 아무도 이 슬라이드를 열어 보지 않았고, 튜닝 없이 최종 설정 그대로 채점했습니다.

성과

77.3%	2.4	16 ms	2,078개
포인팅 단계 점등	발표당 잘못된 하이라이트	리랭커 판단 1회당	채점한 포인팅 단계
합성 음성 · 발표 자료 7종	합성 음성 · 발표 21회	CPU 1스레드	처음 보는 발표 자료 7종

위 포인터 점수는 작성된 스크립트를 합성 음성(TTS)으로 읽어 측정했습니다. 실제 발표는 아직 측정하지 않았습니다.

실제 음성의 단어 오류율(WER)로 선정한 음성 인식 모델	단어 오류율
Canary 180M Flash (선정)	**11.3%**
Whisper large-v3-turbo	32.1%

한계: 모든 포인터 점수는 합성 음성(TTS) 기준이며, 실제 발표는 아직 측정하지 않았습니다. 영어만 지원합니다. 슬라이드 맵 작성에는 발표 자료를 준비할 때 한 번 네트워크가 필요하며, 오프라인으로 동작하는 것은 라이브 루프뿐입니다.

기술 스택

컴퓨터 비전	Object Detection, Semantic / Instance Segmentation, OCR, Multi-Object Tracking, Face Recognition (ArcFace), Pose Estimation
ML / DL 프레임워크	PyTorch, TensorFlow, Keras, Scikit-Learn, Ultralytics (YOLO), OpenCV, InsightFace, Hugging Face Transformers, SAM2
모델 최적화	Quantization, Pruning, TensorRT, ONNX Runtime, DLA 오프로딩, 백본 교체 및 A/B 평가
엣지 하드웨어	NVIDIA Jetson (Orin NX · Orin Nano), Hailo-8 NPU, RK3588 NPU (Orange Pi 5), Raspberry Pi 4/5, Android 태블릿
배포	Docker, FastAPI, Flask, REST API, PostgreSQL, SQLite, 다중 스트림 동시 서빙
평가 · 모니터링	클래스별 지표, 적대적 강건성 테스트, Grad-CAM, FPS/지연/지터 벤치마크, W&B, MLflow, TensorBoard
LLM · 에이전트	Claude API (구조화 출력, 프롬프트 캐싱, Batch API), LLM 평가, 음성 인식 (sherpa-onnx), Cross-Encoder 리랭킹
언어 · 도구	Python (주력), C++, SQL, Linux (Ubuntu), Git/GitHub, GitLab, GitHub Actions CI

학력 · 어학 · 비자

학력

컴퓨터공학 학사	우송대학교 · 대전광역시 · 2019 – 2023

비자

E-7 특정활동	국내 취업 활동 가능

어학

우즈베크어	모국어
영어	업무상 소통 가능 (Fluent)
한국어	일상 회화 가능
러시아어	일상 회화 가능

멘토 · 우송대학교 캡스톤 프로젝트 · 2024

컴퓨터 비전 팀(영상처리 · YOLO 모델 학습)을 주간 피드백으로 지도, 200개 이상 참가 팀 중 1위 달성에 기여

노디르벡 마흐투모프

AI/ML 엔지니어 · 컴퓨터 비전 · 엣지 AI · 모델 최적화

stark.developer2877@gmail.com · 010-5751-0750
github.com/Nodir0705 · linkedin.com/in/nodirbek-makhtumov-03084a194
nodir0705.github.io